

Consignes générales pour les TDs

FAITES BIEN LES SCHEMAS DEMANDES, **CE N'EST PAS OPTIONNEL** ET VOUS AIDERA À VISUALISER ET COMPRENDRE LES ALGORITHMES

POUR LES EXERCICES COMPORTANT DE LA REDACTION DE CODE EN C, **UTILISEZ UN EDITEUR DE TEXTE OU UN IDE**, AFIN DE POUVOIR RECUPERER VOS TRAVAUX POUR LES TPs.

Les exercices précédés d'un symbole losange ◆ sont obligatoires.

TD1 : rappels sur les listes

Listes simplement chaînées – objectif : comprendre les structures utilisées, faire des schémas avec les pointeurs

Rappel : comme indiqué en cours, on utilisera, pour les exercices, les types :

```
struct s_cell
{
    int value;
    struct s_cell *next;
};

typedef struct s_cell t_cell;

et

typedef struct s_list
{
    t_cell *head;
} t_list;
```

◆ Exercice 1) Visualisation de listes

Faites les schémas correspondants aux définitions de listes suivantes :

- a) `t_list list_1 ;` // définition de variable sans initialisation
- b) `t_list list_2 ;`
`list_2 = createList() ;` // initialisation avec liste vide
- c) `t_list list_3 ;`
`list_3 = createList() ;`
`addCell(&list_3, 4)`
`addCell(&list_3, -17)`
`addCell(&list_3, 0)` // ajout en tête de liste de 3 valeurs

◆ Exercice 2) Complément de cours – recherche de valeur dans une liste

Soit une liste nommée **mylist**, et définie ainsi :

`t_list mylist ;`

Cette liste stocke des entiers, et elle a été initialisée.

- a) Ecrivez la fonction qui permet d'indiquer si une valeur donnée est stockée dans la liste. Cette fonction doit fournir un résultat de type True (1) / False (0)

- b) Modifier cette fonction de manière que le résultat soit l'adresse de la cellule qui stocke la valeur cherchée. Quelle valeur doit-on retourner si la valeur n'est pas présente ?

◆ *Exercice 3) Recherche récursive de valeur*

Ecrivez la fonction de recherche de valeur dans une liste en utilisant la récursivité (une fonction pour les cellules, une pour l'appel initial avec la liste)

Rappel : il y a 3 cas à tester

- 1) La liste est vide
- 2) La cellule courante stocke la valeur recherchée
- 3) La cellule courante ne stocke pas la valeur recherchée

◆ *Exercice 4) Complément de cours : détruire/libérer une liste*

Soit la fonction suivante :

```
void destroyList(t_list mylist)
{
    t_cell *curr ;
    curr = mylist.head ;

    while (curr != NULL)
    {
        free(curr) ;
        curr = curr->next;
    }
}
```

- 1) Indiquez pourquoi cette fonction ne fonctionne pas (le programme associé 'plante')
- 2) Modifiez cette fonction pour la rendre fonctionnelle – vous pouvez utiliser une variable supplémentaire
- 3) Ecrivez une version récursive de cette fonction (une fonction pour les cellules, une pour l'appel initial avec la liste)
- 4) Une fois cette fonction effectuée, quelle devrait être la valeur du champ **head** de la liste ?

◆ *Exercice 5) Modification de liste*

On souhaite écrire une fonction itérative nommée **flipVals()**, à qui l'on transmet une liste **mylist**, qui ajoute 1 à toutes les valeurs paires et enlève 1 à toutes les valeurs impaires().

- 1) Cette fonction modifie-t-elle la liste ? (Attention on parle bien de la variable **mylist**)
- 2) Ecrivez cette fonction

◆ *Exercice 6) Suppression dans une liste*

Écrire une fonction nommée **deleteVal()**, qui doit supprimer la première occurrence d'une valeur dans une liste, si elle s'y trouve.

- 1) Cette fonction est-elle susceptible de modifier la liste ? donnez un exemple pour justifier votre réponse
- 2) Ecrivez cette fonction, en utilisant celle de l'exercice 2 ou 3 (recherche de valeur)

◆ *Exercice 7) Réutilisation de fonction*

Écrire une fonction nommée `removeAllValFromList()` à qui l'on fournit une liste (à modifier) et une valeur, et qui supprime **toutes les occurrences** de la valeur dans la liste.

Consigne : il faut réutiliser les fonctions des exercices 2/3 et 6

Exercice 8) fonctions récursives

- 1) Écrire en langage C une fonction récursive `int checkOrderedCellRec(t_cell*)` ; qui indique si les éléments rangés dans les cellules de la liste sont classés en ordre croissant.
- 2) Écrire une fonction `int checkOrderedListRec(t_list)` ; qui utilise `checkOrderedCellRec()` pour faire le premier appel récursif.